

Module 4: Developing IOT

Developing IOT
IoT Platforms Design Methodology: Introduction, IoT Design Methodology, -Case Study on IoT System for Weather Monitoring.
IoT Systems Logical Design using Python, Python Packages of Interest for IoT.
Textbooks:
Internet of Things: A Hands-On Approach, by Arshdeep Bahga & Vijay Madisetti, Universities Press, 2014
Reference books:
1. Introduction to IoT, by Sudip Misra, Anandarup Mukherjee, Arijit Roy, Cambridge University Press 2021 2. IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things – David Hanes, Gonzalo Salgueiro, Patrick Grossetete, Pearson Education, 2017

IOT PLATFORMS DESIGN METHODOLOGY:

INTRODUCTION

1. Introduction to IoT Platforms

- **Definition:** The Internet of Things (IoT) is a network of physical devices (sensors, actuators, controllers, machines, vehicles, etc.) that collect and exchange data over the internet.
- **IoT Platform:** A platform that enables **integration of devices, data management, communication, and application development.**

The Internet of Things (IoT) refers to a system of interconnected physical devices such as sensors, actuators, machines, vehicles, and appliances that are embedded with electronics, software, and communication technologies. These devices collect data from the environment and exchange it over the internet to provide useful services. An IoT platform acts as the middleware that connects devices, manages data, enables communication, and supports application development.

An IoT system is generally structured into six layers. The **Perception Layer** is the lowest layer, consisting of sensors, actuators, RFID tags, and cameras that sense physical parameters such as temperature, humidity, motion, and images. The **Network Layer** ensures connectivity using technologies like Wi-Fi, ZigBee, 4G/5G, and Bluetooth, transmitting sensed data to processing systems. The **Edge Layer** processes data closer to the devices using gateways or edge processors, reducing latency and bandwidth consumption. The **Middleware Layer** provides services such as device management, data storage, and API support to allow applications to use data easily. The

Application Layer is the visible part for users, consisting of mobile apps and dashboards that display IoT insights, such as a weather app or a smart home application. Finally, the **Business Layer** uses the processed data to support decision-making, policy design, and strategies, such as predictive maintenance in industries or yield improvement in agriculture.

2. Layers of IoT Platform:

1. Perception Layer

- The lowest layer (sensing layer).
- Includes sensors, RFID tags, cameras, GPS devices.
- Function: Collects real-world data like temperature, pressure, motion, images.

2. Network Layer

- Provides connectivity using Wi-Fi, Bluetooth, ZigBee, 4G/5G, LoRaWAN.
- Function: Transmits data securely from sensors to servers/cloud.

3. Edge Layer

- Local data processing near the source (edge devices, gateways).
- Function: Reduces latency and bandwidth usage.

4. Middleware Layer

- Acts as a **software bridge** between hardware and applications.
- Function: Data storage, device management, APIs, analytics.

5. Application Layer

- End-user interface: dashboards, mobile apps, monitoring tools.
- Examples: Smart home app, hospital patient dashboard, smart farming app.

6. Business Layer

- Responsible for **decision-making** and overall business logic.
- Examples: Predictive maintenance reports, energy usage optimization, crop yield planning.

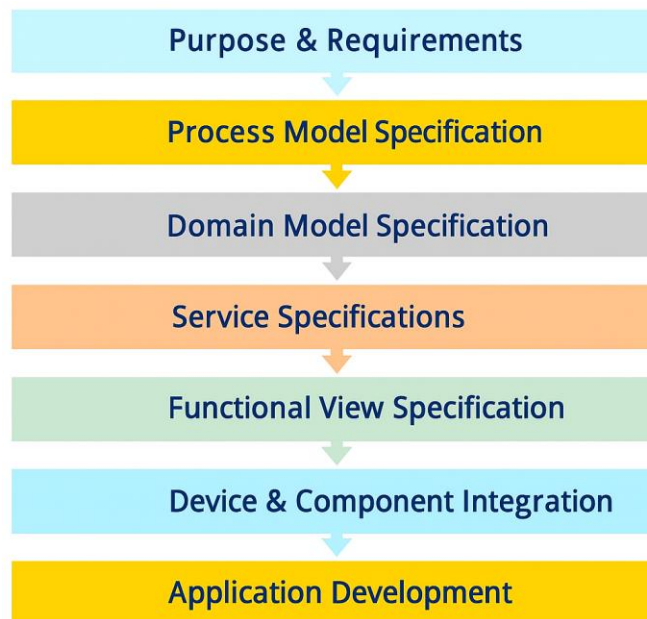
Key Components of IoT Platforms

- **Device Management:** Registration, authentication, configuration, and monitoring of IoT devices
- **Connectivity Management:** Handling various communication protocols (WiFi, Bluetooth, LoRaWAN, 5G)
- **Data Processing:** Real-time and batch processing of sensor data
- **Analytics & Intelligence:** Machine learning, AI, and advanced analytics capabilities
- **Application Enablement:** APIs, SDKs, and development tools
- **Security:** End-to-end encryption, identity management, and threat detection

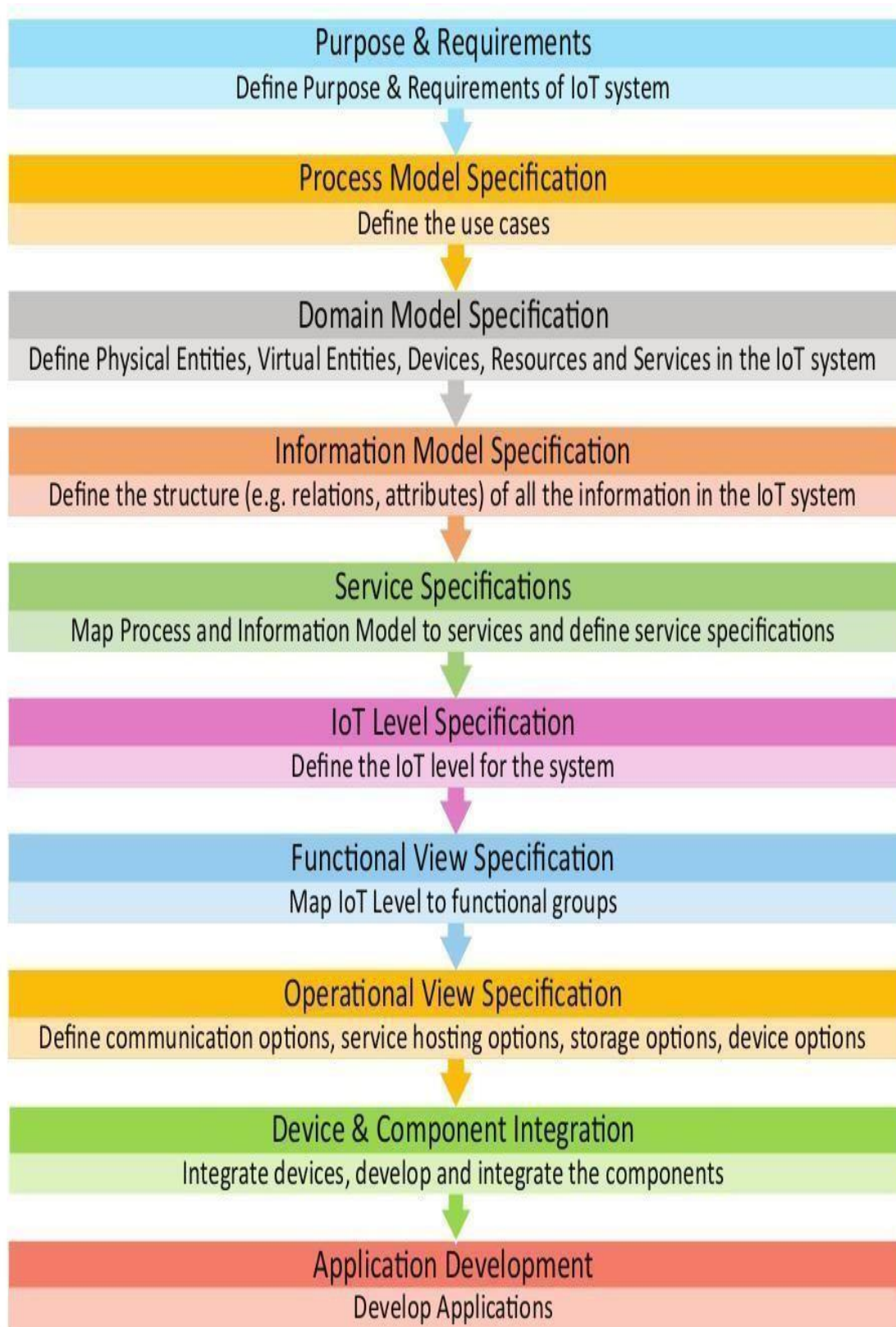
IOT PLATFORMS DESIGN METHODOLOGY

IoT System Design Methodology

The IoT system design methodology follows a systematic ten-step approach that guides developers from initial concept to final implementation, ensuring comprehensive planning and structured development of Internet of Things solutions.



Methodology:



Foundation and Planning Phase:

Step 1: Purpose & Requirements Specification - The first step in IoT system design methodology is to define the purpose and requirements of the system. In this step, the system purpose, behavior and requirements (such as data collection requirements, data analysis requirements, system management requirements, data privacy and security requirements, user interface requirements, etc.) are captured.

Step 2: Process Specification - The second step in the IoT design methodology is to define the process specification. In this step, the use cases of the IoT system are formally described based on and derived from the purpose and requirement specifications.

Step 3: Domain Model Specification - The third step in the IoT design methodology is to define the Domain Model. The domain model describes the main concepts, entities and objects in the domain of IoT system to be designed. Domain model defines the attributes of the objects and relationships between objects. Domain model provides an abstract representation of the concepts, objects and entities in the IoT domain, independent of any specific technology or platform. With the domain model, the IoT system designers can get an understanding of the IoT domain for which the system is to be designed.

System Architecture and Design Phase:

Step 4: Information Model Specification - The fourth step in the IoT design methodology is to define the Information Model. Information Model defines the structure of all the information in the IoT system, for example, attributes of Virtual Entities, relations, etc. Information model does not describe the specifics of how the information is represented or stored. To define the information model, we first list the Virtual Entities defined in the Domain Model. Information model adds more details to the Virtual Entities by defining their attributes and relations.

Step 5: Service Specifications - The fifth step in the IoT design methodology is to define the service specifications. Service specifications define the services in the IoT system, service types, service inputs/output, service endpoints, service schedules, service preconditions and service effects.

Step 6: IoT Level Specification - The sixth step in the IoT design methodology is to define the IoT level for the system.

Step 7: Functional View Specification - The seventh step in the IoT design methodology is to define the Functional View. The Functional View (FV) defines the functions of the IoT systems grouped into various Functional Groups (FGs). Each Functional Group either provides functionalities for interacting with instances of concepts defined in the Domain Model or provides information related to these concepts.

Implementation and Deployment Phase:

Step 8: Operational View Specification - The eighth step in the IoT design methodology is to define the Operational View Specifications. In this step, various options pertaining to the IoT system deployment and operation are defined, such as service hosting options, storage options, device options, application hosting options, etc.

Step 9: Device & Component Integration - The ninth step in the IoT design methodology is the integration of the devices and components.

Step 10: Application Development - The final step in the IoT design methodology is to develop the IoT application.

Designing an IoT system requires a structured methodology that ensures the solution is practical, scalable, and efficient. The first step is **Purpose and Requirements Specification**, which involves clearly defining the problem the IoT solution will solve, such as monitoring weather or automating irrigation in agriculture. Next is the **Process Specification**, where the different workflows are broken down, for example, how data flows from sensors to applications.

The **Domain Model Specification** focuses on identifying and representing real-world entities within the IoT system, such as a temperature sensor or an actuator controlling irrigation. Once the entities are identified, the **Information Model Specification** defines how data is structured and exchanged, such as representing temperature readings in JSON or XML format. The **Service**

Specification stage defines the services the IoT system will provide, for example, a weather forecasting API or an irrigation alert notification service.

The **IoT Level Specification** step involves deciding at what level the system will operate—whether at the device-only level, device plus gateway, or cloud-based IoT level. The **Functional View Specification** highlights the required functions such as sensing, storing, analyzing, and alerting. On the other hand, the **Operational View Specification** deals with real-world deployment details such as where the devices will be placed, how they will be powered, and what communication medium they will use. After this, **Device and Component Integration** brings together hardware (sensors, actuators) and software (protocols, APIs, databases) into a single ecosystem. Finally, **Application Development** creates the user-facing applications, such as a mobile dashboard or monitoring website, completing the IoT design process.

Case Studies:

A practical example of IoT is the **Weather Monitoring System**. In this system, IoT-enabled sensors measure parameters such as temperature, humidity, pressure, and rainfall. These values are transmitted to a server using REST APIs and stored in a database. A web or mobile application then retrieves and visualizes the information, allowing real-time monitoring of weather conditions. Such a system is vital for accurate weather forecasting, disaster management, and supporting agricultural planning.

Another important case is **Smart Agriculture**. Farmers use IoT sensors such as soil moisture detectors, pH sensors, and crop health cameras to monitor field conditions. This data enables precision farming, where irrigation, fertilization, and pesticide use are automated based on real-time requirements. Drones can also be used to spray pesticides and capture aerial images of crop health. The result is improved productivity, reduced costs, and sustainable farming practices.

A third example is **Industrial Monitoring**. In factories, IoT devices monitor machine health by measuring vibration, temperature, and energy consumption. If anomalies are detected, predictive maintenance can be scheduled to prevent costly breakdowns. This improves safety, reduces downtime, and optimizes resource usage, making industries more efficient.

CASE STUDY: IOT WEATHER MONITORING SYSTEM

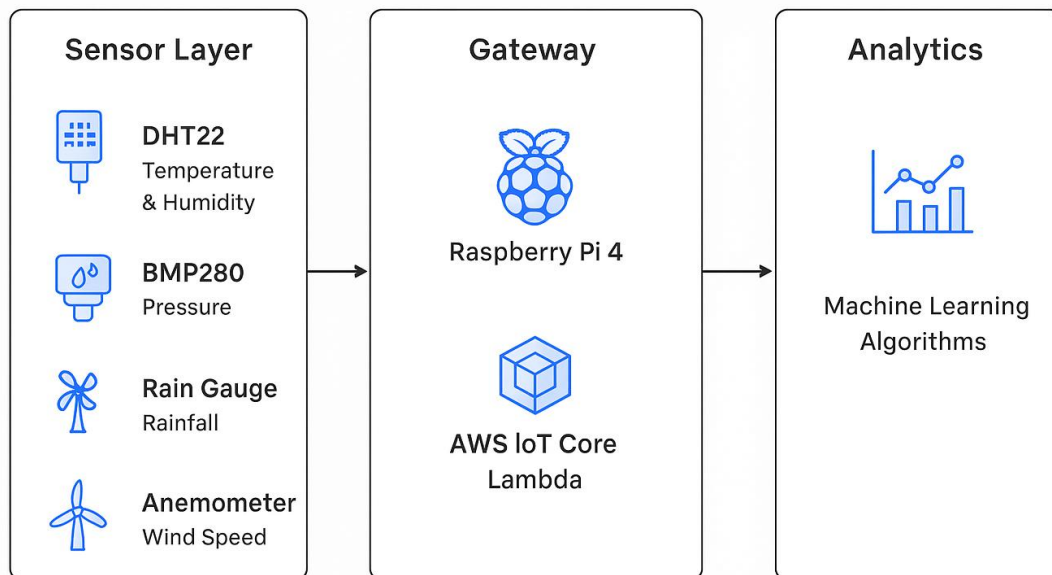
System Overview:

A comprehensive weather monitoring system that collects environmental data from multiple sensors and provides real-time weather information and forecasting capabilities.

System Requirements:

- Monitor temperature, humidity, pressure, wind speed, and rainfall
- Real-time data collection and transmission
- Historical data storage and analysis
- Weather alerts and notifications
- Web and mobile dashboard access
- Data accuracy within $\pm 2\%$ tolerance

IoT WEATHER MONITORING — BLOCK DIAGRAM



The IoT-based weather monitoring system continuously measures environmental parameters like **temperature, humidity, gas, and pressure**, and transmits them to a cloud server for **real-time visualization and alerting**.

It ensures **remote monitoring, data logging, and predictive analysis** through cloud-based IoT platforms such as **Ubidots, ThingSpeak, or AWS IoT**.

1. Sensor Layer

This is the **foundation** of the system. It includes multiple sensors that continuously collect real-time environmental data:

- **DHT22:** Measures **temperature** and **humidity**.
- **BMP280:** Measures **atmospheric pressure**.
- **Rain Gauge:** Detects **rainfall amount or intensity**.
- **Anemometer:** Measures **wind speed**.

Each sensor sends its data to the microcontroller or gateway through digital or analog signals.

2. Gateway Layer

The **Raspberry Pi 4** acts as the **gateway**, serving as a bridge between the sensor network and the cloud.

- It collects all sensor data via **wired (I2C, UART)** or **wireless (LoRa)** communication.
- The gateway supports **dual connectivity**:
 - **Wi-Fi:** For local internet access.
 - **LoRa:** For long-range, low-power transmission in remote areas.
- Data is **pre-processed, buffered, and encrypted** before uploading to the cloud.

3. Cloud Platform Layer

This layer handles **data ingestion, processing, and storage** using **AWS IoT Core** and **AWS Lambda**.

- **AWS IoT Core:** Manages incoming MQTT data streams securely from the gateway.
- **AWS Lambda:** Performs **serverless data processing** — validation, formatting, and triggering alerts.
- Processed data is stored in **AWS S3** or **DynamoDB** for long-term analysis.

4. Analytics & Machine Learning Layer

This layer performs data analysis and prediction using stored historical and real-time data.

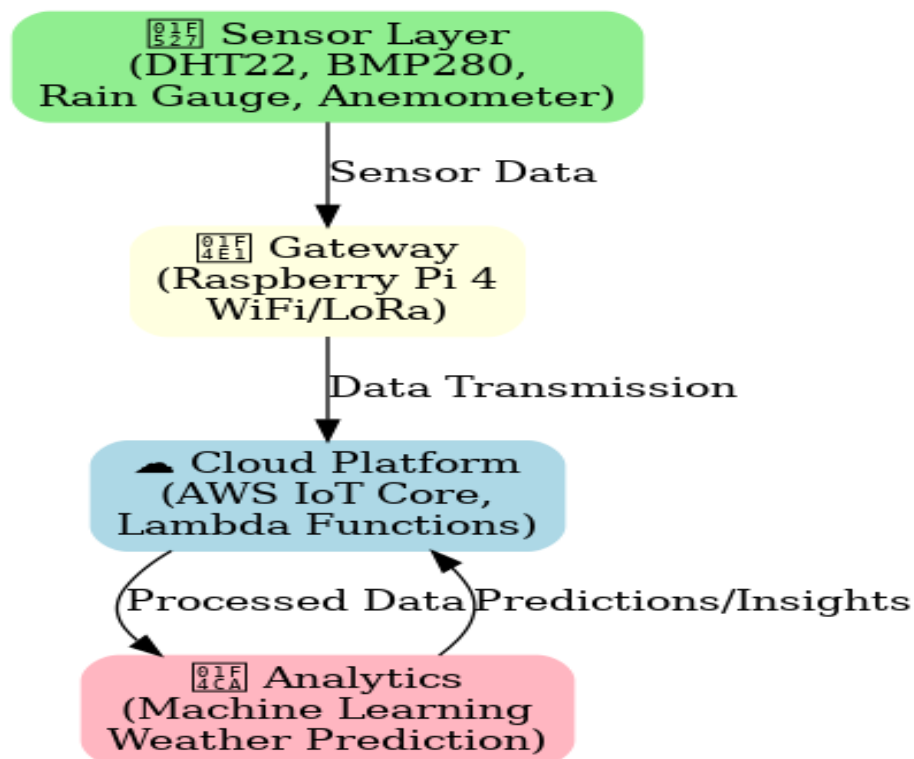
- **Machine Learning Models** analyze trends and predict weather conditions like rain probability or wind patterns.
- **Anomaly detection algorithms** identify sudden environmental changes.
- Outputs are used to generate **forecasts, warnings, or automated alerts**.

5. Dashboard & User Access Layer

The processed and analyzed data is visualized for users through:

- **Web Dashboard:** Displays live data graphs, weather trends, and device status.
- **Mobile Application:** Sends alerts, notifications, and daily summaries to users.

Architecture Components:



This diagram illustrates a comprehensive IoT weather monitoring system architecture with four distinct layers working together to collect, transmit, process, and analyze environmental data.

The **Sensor Layer** forms the foundation of the system, incorporating multiple environmental sensors including the DHT22 for temperature and humidity measurements, the BMP280 for barometric pressure sensing, a rain gauge for precipitation monitoring, and an anemometer for wind speed detection. These sensors work continuously to gather real-time environmental data from the deployment location.

The **Gateway** component serves as the critical communication bridge, utilizing a Raspberry Pi 4 equipped with both WiFi and LoRa (Long Range) connectivity options. This dual connectivity approach ensures reliable data transmission even in remote locations where traditional WiFi networks may be unavailable, with LoRa providing long-range, low-power communication capabilities for extended battery life and coverage.

The **Cloud Platform** leverages AWS IoT Core integrated with Lambda functions to provide scalable, serverless data processing and storage. AWS IoT Core handles the secure ingestion and management of sensor data streams, while Lambda functions enable real-time data processing, transformation, and automated responses to specific environmental conditions without the need for dedicated server infrastructure.

Finally, the **Analytics** layer employs machine learning algorithms specifically designed for weather prediction and pattern recognition. This component analyzes historical and real-time data to generate forecasts, identify weather trends, detect anomalies, and provide predictive insights that can be valuable for agricultural applications, environmental monitoring, or early warning systems for severe weather conditions.

IoT Systems Logical Design using Python:

Why Python for IoT?

- **Simplicity:** Easy syntax and rapid development capabilities
- **Rich Ecosystem:** Extensive libraries for IoT development
- **Hardware Support:** GPIO control, I2C, SPI communication
- **Data Processing:** Pandas, NumPy for data analysis
- **Machine Learning:** TensorFlow, PyTorch for AI integration
- **Networking:** Built-in networking and protocol support
- Python has emerged as the preferred programming language for IoT development due to its unique combination of accessibility and powerful capabilities that align perfectly with the demands of Internet of Things applications. The language's **simplicity** is perhaps its greatest asset, offering an intuitive syntax that enables developers to write clean, readable code quickly, making it ideal for rapid prototyping and iterative development cycles that are common in IoT projects where requirements often evolve rapidly.
- The **rich ecosystem** surrounding Python provides developers with an extensive collection of specialized libraries and frameworks specifically designed for IoT development, eliminating the need to build complex functionality from scratch and significantly accelerating project timelines. This ecosystem includes comprehensive **hardware support** through libraries that enable direct control of GPIO pins, I2C communication protocols, and SPI interfaces, making it straightforward to interface with sensors, actuators, and other electronic components commonly found in IoT systems.
- Python's strength in **data processing** is particularly valuable for IoT applications that generate continuous streams of sensor data, with libraries like Pandas and NumPy providing sophisticated tools for data manipulation, cleaning, analysis, and visualization that are essential for extracting meaningful insights from raw sensor readings. The language's **machine learning** capabilities through frameworks like TensorFlow and PyTorch enable developers to integrate artificial intelligence directly into IoT systems, allowing for edge computing scenarios where devices can make intelligent decisions locally without relying on cloud connectivity.

- Finally, Python's robust **networking** capabilities include built-in support for various communication protocols commonly used in IoT environments, from HTTP and MQTT for cloud connectivity to more specialized protocols for device-to-device communication, making it possible to create comprehensive IoT solutions that can communicate seamlessly across different network architectures and connectivity options.

The main characteristics of Python are:

- **Multi-paradigm programming language**
Python supports more than one programming paradigms including object-oriented programming and structured programming
- **Interpreted Language**
Python is an interpreted language and does not require an explicit compilation step. The Python interpreter executes the program source code directly, statement by statement, as a processor or scripting engine does.
- **Interactive Language**
Python provides an interactive mode in which the user can submit commands at the Python prompt and interact with the interpreter directly.

Python – Benefits

- **Easy-to-learn, read and maintain**
Python is a minimalistic language with relatively few keywords, uses English keywords and has fewer syntactical constructions as compared to other languages. Reading Python programs feels like English with pseudo-code like constructs. Python is easy to learn yet an extremely powerful language for a wide range of applications.
- **Object and Procedure Oriented**
Python supports both procedure-oriented programming and object-oriented programming. Procedure oriented paradigm allows programs to be written around procedures or functions that allow reuse of code. Procedure oriented paradigm allows programs to be written around objects that include both data and functionality.
- **Extendable**
Python is an extendable language and allows integration of low-level modules written in

languages such as C/C++. This is useful when you want to speed up a critical portion of a program.

- **Scalable**

Due to the minimalistic nature of Python, it provides a manageable structure for large programs.

- **Portable**

Since Python is an interpreted language, programmers do not have to worry about compilation, linking and loading of programs. Python programs can be directly executed from source.

- **Broad Library Support**

Python has a broad library support and works on various platforms such as Windows, Linux, Mac, etc.

Installing Python – Setup

Windows

- Python binaries for Windows can be downloaded from <http://www.python.org/getit> .
- For the examples and exercise in this book, you would require Python 2.7 which can be directly downloaded from: <http://www.python.org/ftp/python/2.7.5/python-2.7.5.msi>
- Once the python binary is installed you can run the python shell at the command prompt using `> python`

Modules

- Python allows organizing the program code into different modules which improves the code readability and management.
- A module is a Python file that defines some functionality in the form of functions or classes.
- Modules can be imported using the import keyword.
- Modules to be imported must be present in the search path.

Packages

- Python package is hierarchical file structure that consists of modules and subpackages.
- Packages allow better organization of modules related to a single application environment.

File Handling

- Python allows reading and writing to files using the file object.
- The open (filename, mode) function is used to get a file object.
- The mode can be read (r), write (w), append (a), read and write (r+ or w+), read-binary (rb), write-binary (wb), etc.

Example of reading a certain number of bytes

```
>>>fp = open('file.txt','r')
>>>fp.read(10) 'Python sup'
>>>fp.close()
```

Example of getting the current position of read

```
>>>fp = open('file.txt','r')
>>>fp.read(10) 'Python sup'
>>>currentpos = fp.tell
>>>print currentpos
<built-in method tell of file object at 0x0000000002391390>
>>>fp.close()
```

Classes

- Python is an Object-Oriented Programming (OOP) language. Python provides all the standard features of Object-Oriented Programming such as classes, class variables, class methods, inheritance, function overloading, and operator overloading.

Class

- A class is simply a representation of a type of object and user-defined prototype for an object that is composed of three things: a name, attributes, and operations/ methods.

Instance/Object

- Object is an instance of the data structure defined by a class.

Inheritance

- Inheritance is the process of forming a new class from an existing class or base class.

Function overloading

- Function overloading is a form of polymorphism that allows a function to have different meanings, depending on its context.

Operator overloading

- Operator overloading is a form of polymorphism that allows assignment of more than one function to a particular operator.

Function overriding

Function overriding allows a child class to provide a specific implementation of a function that is already provided by the base class. Child class implementation of the overridden function has the same name, parameters and return type as the function in the base class.

Class Example

The variable studentCount is a class variable that is shared by all instances of the class Student and is accessed by Student.studentCount.

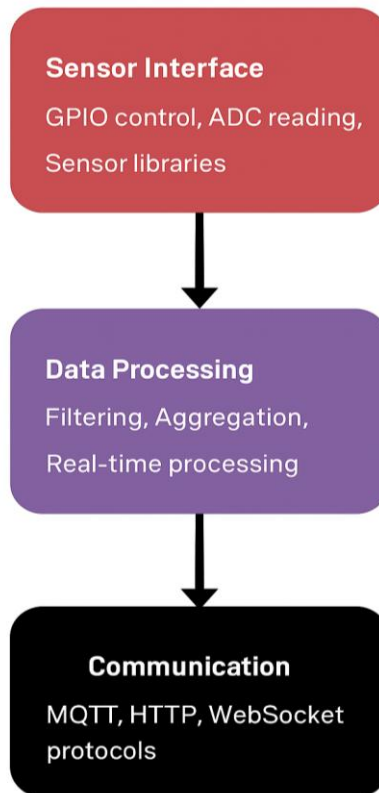
The variables name, id and grades are instance variables which are specific to each instance of the class.

Class Inheritance

- In this example Shape is the base class and Circle is the derived class. The class Circle inherits the attributes of the Shape class.
- The child class Circle overrides the methods and attributes of the base class (eg. draw() function defined in the base class Shape is overridden in child class Circle).
- There is a special method by the name init () which is the class constructor. The class constructor initializes a new instance when it is created. The function del() is the class destructor.
- # Examples of class inheritance class Shape:
def init (self):
print "Base class constructor" self.color = 'Green' self.lineWeight = 10.0
def draw(self):

```
print "Draw - to be implemented" def setColor(self, c):  
    self.color = c
```

Logical Design Components:

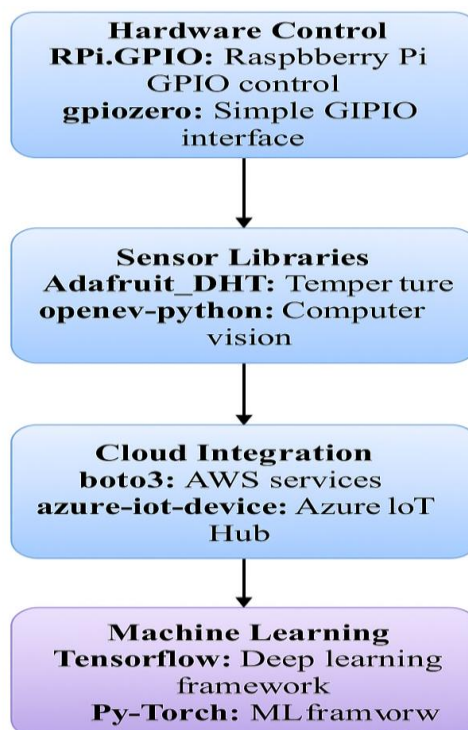


This architecture diagram illustrates the core software components of a Python-based IoT system, demonstrating how different functional layers work together to create a comprehensive sensor data collection and transmission solution. The **Sensor Interface** component forms the hardware abstraction layer, utilizing Python's GPIO control capabilities to directly interface with physical sensors, ADC (Analog-to-Digital Converter) reading functionality to convert analog sensor signals into digital data, and specialized sensor libraries that provide pre-built drivers and methods for common IoT sensors, simplifying the process of data acquisition from various environmental monitoring devices.

The **Data Processing** layer represents the analytical heart of the system, implementing sophisticated algorithms for filtering raw sensor data to remove noise and anomalies, performing aggregation operations to summarize data over time periods or combine readings from multiple sensors, and executing real-time processing routines that can make immediate decisions based on incoming sensor values. This layer leverages Python's powerful data processing libraries like Pandas and NumPy to efficiently handle large volumes of sensor data, perform statistical analysis, and prepare information for transmission or local storage.

The **Communication** component serves as the connectivity backbone, implementing multiple protocol support including MQTT for lightweight, publish-subscribe messaging ideal for IoT scenarios, HTTP for standard web-based communication with cloud services and APIs, and WebSocket protocols for real-time, bidirectional communication channels. This multi-protocol approach ensures the system can adapt to various network conditions and requirements, whether communicating with local gateways, cloud platforms, or other IoT devices, while Python's built-in networking libraries and extensive third-party protocol implementations make it straightforward to establish reliable, secure connections across different communication mediums.

Python Packages of Interest for IoT:



This comprehensive overview showcases the essential Python packages that form the backbone of modern IoT development, organized into six critical functional categories that address every aspect of Internet of Things system implementation. The **Hardware Control** category features foundational libraries like RPi.GPIO for direct Raspberry Pi GPIO pin manipulation and gpiozero, which provides a simplified, more intuitive interface for controlling hardware components, making it easier for developers to interact with sensors, motors, LEDs, and other electronic components without dealing with low-level hardware complexities.

The **Sensor Libraries** section highlights specialized packages such as Adafruit_DHT for interfacing with popular temperature and humidity sensors, and opencv-python for computer vision applications, enabling IoT devices to process visual data from cameras and implement intelligent image recognition capabilities. These libraries abstract away the intricate details of sensor communication protocols and provide ready-to-use functions for data acquisition from various sensing devices.

Communication packages are essential for IoT connectivity, with paho-mqtt providing a robust MQTT client implementation for lightweight, efficient messaging between IoT devices and cloud services, while the requests library offers comprehensive HTTP client functionality for RESTful API interactions and web service communication. These tools ensure seamless data transmission across different network architectures and protocols.

The **Data Processing** category leverages Python's analytical strengths through pandas for sophisticated data manipulation, cleaning, and analysis operations, complemented by numpy for high-performance numerical computing and mathematical operations on large datasets. The **Cloud Integration** section features boto3 for seamless AWS services integration and azure-iot-device for Microsoft Azure IoT Hub connectivity, enabling scalable cloud-based IoT solutions.

Finally, the **Machine Learning** category incorporates cutting-edge AI frameworks including TensorFlow for deep learning applications and PyTorch for flexible machine learning model development, allowing IoT systems to implement intelligent edge computing capabilities, predictive analytics, and automated decision-making processes directly on IoT devices or through cloud-based AI services.

Python for IoT Systems

Python is one of the most widely used programming languages in IoT systems due to its simplicity and versatility. Its minimal syntax makes it beginner-friendly, while its wide support on platforms like Raspberry Pi and MicroPython makes it ideal for small embedded boards. Furthermore, Python has a large ecosystem of libraries that support IoT applications ranging from hardware communication to artificial intelligence and data visualization.

Python offers features that are particularly useful in IoT. Its **data types** such as numbers, strings, lists, tuples, and dictionaries allow easy representation of IoT data. Its **control flow structures** (if-else, loops, range, pass) allow developers to define the logic for handling sensor inputs. Using **functions** defined with the def keyword, developers can modularize IoT code for reusability. **Modules and Packages** in Python allow structured programming, which is vital for large IoT projects. Python also supports **file input and output**, which is useful for storing logs or sensor readings. Its **date and time handling** makes it possible to timestamp IoT data accurately. Finally, its support for **Object-Oriented Programming (OOP)** allows IoT devices to be modeled as objects, such as a class representing a temperature sensor.

Python Packages of Interest:

Python offers several packages that make IoT development easier. **NumPy** is used for numerical computations and array handling, which is essential when processing sensor data. **Pandas** is helpful in organizing IoT datasets into tabular structures and performing analysis. **Matplotlib** and **Seaborn** allow developers to visualize IoT data through graphs and charts.

Communication in IoT often uses lightweight protocols, and the **paho-mqtt** package is used to implement the MQTT protocol for device-to-device or device-to-cloud communication. The **requests** package is widely used for making REST API calls, enabling IoT devices to communicate with cloud services. For hardware interaction, **RPi.GPIO** provides access to Raspberry Pi's GPIO pins to control LEDs, motors, or read sensors. **OpenCV** is a powerful library for computer vision applications, such as detecting plant diseases in agriculture or monitoring vehicles in transport systems. For machine learning and artificial intelligence

integration, packages like **TensorFlow**, **PyTorch**, and **Scikit-learn** allow IoT devices to become intelligent by predicting outcomes and automating decisions.

Python Packages of Interest for IoT are:

- JSON
- XML
- HTTPLib & URLLib
- SMTPLib

JSON (Java Script Object Notation):

- Easy to read and write data – interchange format
- JSON is used as an alternative to XML
- Easy for machines to parse and generate
- Built is two structures:
 - i. Collection of names – value pairs (Dictionaries in Python)
 - ii. Ordered list of values (List in Python)

JSON format is used for serializing and transmitting structured data over a N/W connection

Example, transmitting data b/w server and web application.

■ Box 6.35: JSON Example - A Twitter tweet object

```
{
  "created_at":
    "Sat Jun 01 11:39:43 +0000 2013",
  "id":340794787059875841,
  "text":"What a bright and sunny day today!",
  "truncated":false,
  "in_reply_to_status_id":null,
  "user":{
    "id":383825039,
    "name":"Harry",
    "followers_count":316,
    "friends_count":298,
    "listed_count":0,
    "created_at":"Sun Oct 02 15:51:16 +0000 2011",
    "favorites_count":251,
    "statuses_count":1707,
    :
    "notifications":null
  },
  "geo":{
    "type":"Point",
    "coordinates":[26.92782727,75.78908449]
  },
  "coordinates":{
```

```
"type": "Point",
"coordinates": [75.78908449, 26.92782727]
},
"place": null,
"contributors": null,
"retweet_count": 0,
"favorite_count": 0,
"entities": {
  "hashtags": [],
  "symbols": [],
  "urls": [],
  "user_mentions": []
},
"favorited": false,
"retweeted": false,
"filter_level": "medium",
"lang": "nl"
```

Exchange of information encoded as JSON involves encoding and decoding steps. The Python JSON package provides functions for encoding and decoding JSON. Box 6.36 shows an example of JSON encoding and decoding

■ Box 6.36: Encoding & Decoding JSON in Python

```
>>>import json

>>>message = {
    "created": "Wed Jun 31 2013",
    "id": "001",
    "text": "This is a test message.",
}

>>>json.dumps(message)
'{"text": "This is a test message.", "id": "001",
"created": "Wed Jun 31 2013"}'

>>>decodedMsg = json.loads('{"text": "This is a
test message.", "id": "001", "created": "Wed Jun 31 2013"}')

>>>decodedMsg['created']
u'Wed Jun 31 2013'
>>>decodedMsg['text']
```

XML

- XML (Extensible Markup Language) is a data format for structured document interchange.
- Box 6.37 shows an example of an XML file. In this section you will learn how to parse, read and write XML with Python.
- The Python minidom library provides a minimal implementation of the Document Object Model interface and has an API similar to that in other languages.
- Box 6.38 shows a Python program for parsing an XML file.
- Box 6.39 shows a Python program for creating an XML file.

■ Box 6.37: XML example

```
<?xml version="1.0"?>
<catalog>
  < plant id='1' >
    <common>Bloodroot</common>
    <botanical>Sanguinaria canadensis</botanical>
    <zone>4</zone>
    <light>Mostly Shady</light>
    <price> 2.44 </price>
    <availability>031599</availability>
  </plant>
  <plant id='2' >
    <common>Columbine</common>
    <botanical>Aquilegia canadensis</botanical>
    <zone>3</zone>
    <light>Mostly Shady</light>
    <price> 9.37</price >
    <availability>030699</availability>
  </plant>
  <plant id='3' >
    <common>Marsh Marigold</common>
    <botanical>Caltha palustris</botanical>
    <zone>4</zone>
    <light>Mostly Sunny</light>
    <price> 6.81</price>
    <availability>051799</availability>
  </plant>
</catalog>
```

HTTPLib & URLLib

HTTPLib2 and URLLib2 are Python libraries used in network/internet programming.

HTTPLib2 is an HTTP client library and URLLib2 is a library for fetching URLs.

Box 6.40 shows an example of an HTTP GET request using the **HTTPLib**.

Box 6.40: The variable `resp` contains the response headers and `content` contains the content retrieved from the URL

```
>>> import httpplib2

>>> h = httpplib2.Http()

>>> resp, content = h.request("http://example.com", "GET")

>>> resp

{'status': '200', 'content-length': '1270', 'content-location':
'http://example.com/', 'x-cache': 'HIT', 'accept-ranges': 'bytes',
'server': 'ECS (cpm/FBS/8)', 'last-modified': 'Thu, 25 Apr 2013 16:13:23 GMT',
'etag': '"760602-4f6-4db31b2978ec0"', 'date': 'Wed, 31 Jul 2013 12:36:05 GMT',
'content-type': 'text/html; charset=UTF-8'}

>>> content

'<!doctype html>\n<html>\n<head>\n<title>Example Domain</title>\n\n<meta charset="utf-8">\n'
```

Box 6.41 shows an HTTP request example using URLLib2.

A request object is created by calling `urllib2.Request` with the URL to fetch as input parameter.

Then `urllib2.urlopen` is called with the request object which returns the response object for the requested URL. The response object is read by calling the **read function**.

■ Box 6.41: HTTP request example using URLLib2

```
>>> import urllib2
>>>
>>> req = urllib2.Request('http://example.com')
>>> response = urllib2.urlopen(req)
>>> response_page = response.read()
>>> response_page
'<!doctype html>\n<html>\n<head>\n
<title>Example Domain</title>\n\n
<meta charset="utf-8" />\n'
```

Box 6.4.2 shows an example of an HTTP POST request. The data in the POST body is encoded using the `urlencode` function from `urllib`.

■ Box 6.42: HTTP POST example using HTTPLib2

```
>>> import httplib2
>>> import urllib
>>> h = httplib2.Http()
>>> data = {'title': 'Cloud computing'}
>>> resp, content =
h.request("http://www.htmlcodetutorial.com/cgi-bin/mycgi.pl", "POST",
urllib.urlencode(data))
>>> resp
{'status': '200', 'transfer-encoding': 'chunked',
'server': 'Apache/2.0.64 (Unix) mod_ssl/2.0.64 OpenSSL/0.9.7a
mod_auth_passthrough/2.1 mod_bwlimited/1.4 FrontPage/5.0.2.2635
PHP/5.3.10', 'connection': 'close', 'date': 'Wed, 31 Jul 2013
12:41:20 GMT', 'content-type': 'text/html; charset=ISO-8859-1'}

>>> content
'<HTML>\n<HEAD>\n<TITLE>Idocs Guide to
HTML: My CGI</TITLE>\n</HEAD>
:'
```

6.43 shows an example of sending data to a URL using URLLib2.

■ Box 6.43: Example of sending data to a URL

```
>>> import urllib
>>> import urllib2
>>>
>>> url = 'http://www.htmlcodetutorial.com/cgi-bin/mycgi.pl'
>>> values = ('title' : 'Cloud Computing',
... 'language' : 'Python' )
>>>
>>> data = urllib.urlencode(values)
>>> req = urllib2.Request(url, data)
>>> response = urllib2.urlopen(req)
>>> the_page = response.read()
>>> the_page
'<HTML>\n<HEAD>\n<TITLE>Idocs Guide to
HTML: My CGI</TITLE>\n</HEAD>
```

SMTPLib

Simple Mail Transfer Protocol (SMTP) is a protocol which handles sending email and routing e-mail between mail servers.

The python smtplib module provides an SMtp client session object that can be used to send email.

Python provides a built-in library for interacting with SMTP (Simple Mail Transfer Protocol) servers, which is essential for sending emails. This library is called **smtplib**.

Box 6.44 shows a Python example of sending email from a gmail account.

■ Box 6.44: Python example of sending email

```
import smtplib

from_email = '<enter-gmail-address>'
recipients_list = ['<enter-sender-email>']
cc_list = [ ]
subject = 'Hello'
message = 'This is a test message.'
```



```
username = '<enter-gmail-username>'
password = '<enter-gmail-password>'
server = 'smtp.gmail.com:587'

def sendemail(from_addr, to_addr_list, cc_addr_list,
subject, message,
login, password,
smtpserver):

header = 'From: %s\n' % from_addr
header += 'To: %s\n' % ','.join(to_addr_list)
header += 'Cc: %s\n' % ','.join(cc_addr_list)
header += 'Subject: %s\n\n' % subject
message = header + message

server = smtplib.SMTP(smtpserver)
server.starttls()
server.login(login,password)
problems = server.sendmail(from_addr, to_addr_list, message)
server.quit()

#Send email
sendemail(from_email, recipients_list, cc_list, subject,
message, username, password, server)
```

To send an email, first a connection is established with the SMTP server by calling smtplib.SMTP with the SMTP server name and port.

The user name and password provided are then used to login into the server.the email is then sent by calling server.sendmail function with the from address,to address list and message as input parameters.

Real-World Applications of IoT:

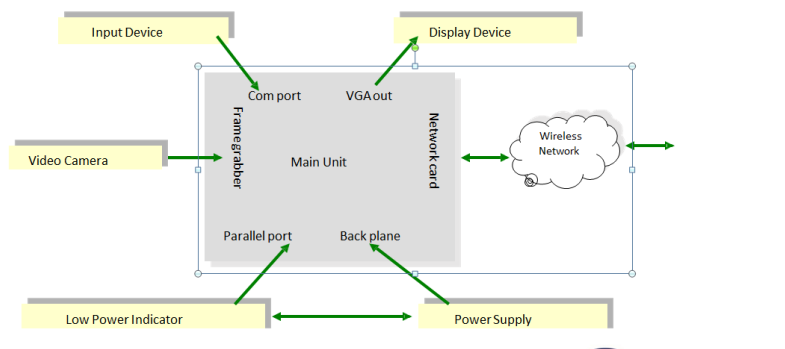
IoT has countless real-world applications. In a **Smart Home**, devices like lights, fans, air conditioners, and security cameras are connected and controlled remotely via mobile apps or voice assistants like Alexa and Google Home. This enhances comfort, energy efficiency, and security.

In **Smart Agriculture**, IoT supports climate condition monitoring, automated irrigation, and drone-based crop management, which help farmers maximize yield while minimizing resource use. **Wearable IoT systems**, such as fitness bands and smartwatches, monitor vital health parameters like heart rate, oxygen levels, and sleep patterns, providing valuable health insights.

In **Healthcare**, IoT enables remote patient monitoring, smart pills that transmit usage data, and connected hospital equipment for better management. In **Connected Cars and Transport**, sensors measure speed, fuel level, and tire pressure, while IoT applications provide accident alerts, navigation, and traffic management.

A **Smart City** integrates IoT into public services such as intelligent streetlights, waste management bins, and air quality monitoring. The smart city development model involves capturing sensor data, processing it, making decisions, and delivering services to citizens, thereby improving quality of life.

IoT Wearable System



IoT Characteristics:

IoT has several unique characteristics. **Interconnectivity** ensures that devices are always connected to the internet and to each other. **Scalability** allows IoT systems to grow from a few devices to millions without losing performance. **Real-time monitoring** enables instant updates, which is critical in applications like healthcare and transportation. **Automation** reduces the need for human intervention by enabling devices to make decisions. Finally, IoT systems exhibit **intelligence** when integrated with artificial intelligence and machine learning, making them capable of self-learning and predictive actions.

Advantages and Disadvantages of IoT:

The advantages of IoT are significant. It enables automation and efficiency, reduces human effort, and lowers costs. Industries save money through predictive maintenance, farmers optimize resource usage, and patients benefit from better healthcare monitoring. IoT also enhances safety and security by providing constant monitoring. However, IoT also comes with disadvantages. Security and privacy risks are major concerns, as billions of connected devices increase the chances of hacking. The initial deployment cost of IoT systems can be high, and interoperability remains a problem because devices from different manufacturers may not work together seamlessly. Additionally, managing and maintaining such complex systems can be challenging.

Advantages of the IoT

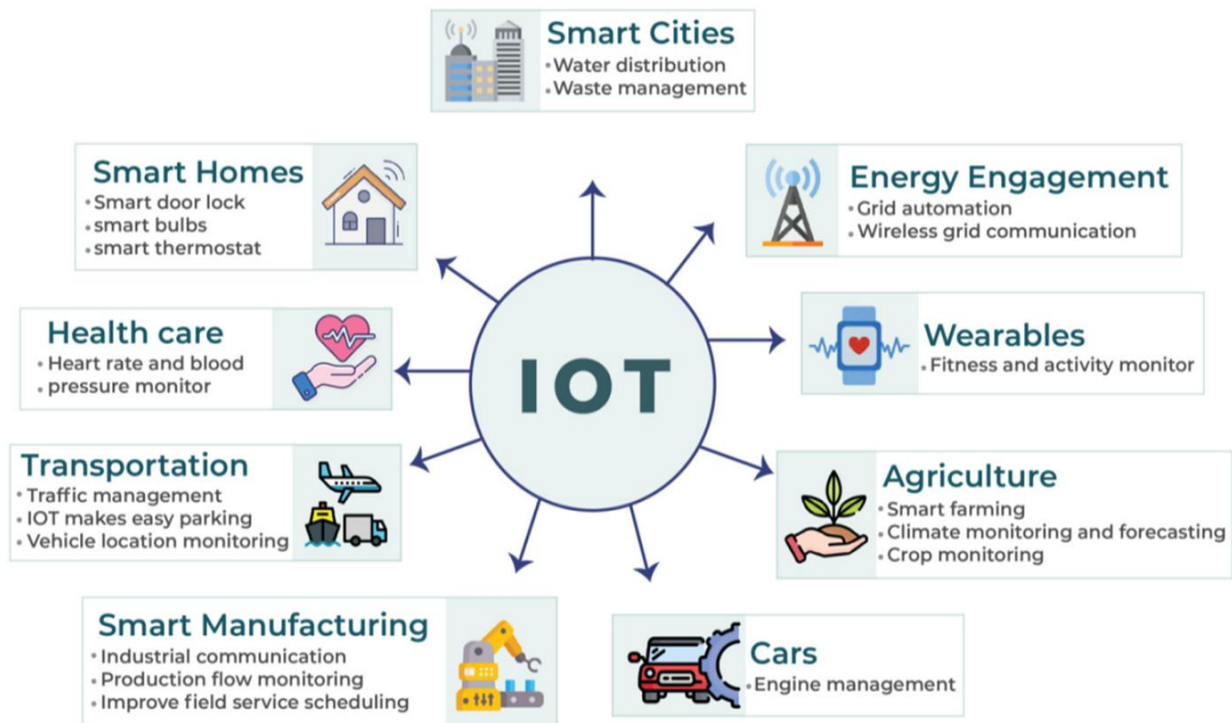
Improved Customer Engagement and Decision making • Transform passive engagement into vibrant and active engagement with users • Gain insight into potential new products and service • Deliver post-sales services efficiently	Efficiency and lower operating costs • Use cheap technology to lower the operating expenses and promote energy conservation • Monitor the performance, quality, and reliability of products and services
Reduced Waste • Make areas of improvement clear • Provide real-world information leading to the more effective management of resources	Technology Optimization • Improve customer experience and better understand product use • Unlock the world of critical functional and field data • Deliver revenue-generating post-sales services
Enhanced Data Collection • Collect sensing data from the right places in the right times • Remote troubleshooting of products	New business opportunities • IoT provides the the ability to collect data from the network and use advanced analytics to uncover new business insights and opportunities

Disadvantages of IoT

Network security IoT creates an ecosystem of connected devices communicating over networks which leave users exposed to various kinds of security attacks	Data privacy The sophistication of IoT provides substantial personal data in extreme detail without the user's active participation
Complexity - IoT systems are complicated in terms of design, deployment, and maintenance - IoT often uses multiple technologies and a large set of new enabling technologies	Flexibility IoT systems are flexible in integrating easily with each other, which may cause conflicts between different vendors or locked systems
Compliance - IoT technology must comply with regulations - IoT complexity makes the issue of compliance seem incredibly challenging	Unemployment - The wide-use of IoT will result in a significant loss of mundane jobs - IoT will have a devastating impact on the employment prospects of less-educated workers

IoT Future Trends

Wearables Ecosystems - Complete programming and application development environments beyond the device level - Wearables as parts of the IoT ecosystem	Interoperability - Across devices of different types and from different vendors - Across different ecosystems - Single entry point for managing personal data	Novel IoT Services - Integrated IoT wearables services combining data and services from multiple ecosystems - Driven by innovation for fitness, healthcare, industry, etc.
---	---	---



How is IoT Shaping Agriculture?

Smart Agriculture Sensors

- E.g. weather conditions, soil quality, crop's growth progress or cattle's health.
- Data can be used to track the state of the agriculture field as well as equipment efficiency.

Cost Management and Waste Reduction

- Monitor anomalies in the crop growth or livestock
- Mitigate the risks of losing the yield
- Lead to higher revenue.

Process Automation

- Increase business efficiency through process automation
- By using smart devices, farmers can automate multiple processes across your production cycle
- E.g. irrigation, fertilizing, or pest control.

Enhanced Product Quality and Volumes

- Achieve better control over the production process
- Maintain higher standards of crop quality and growth capacity through automation.

IoT Applications - Smart City

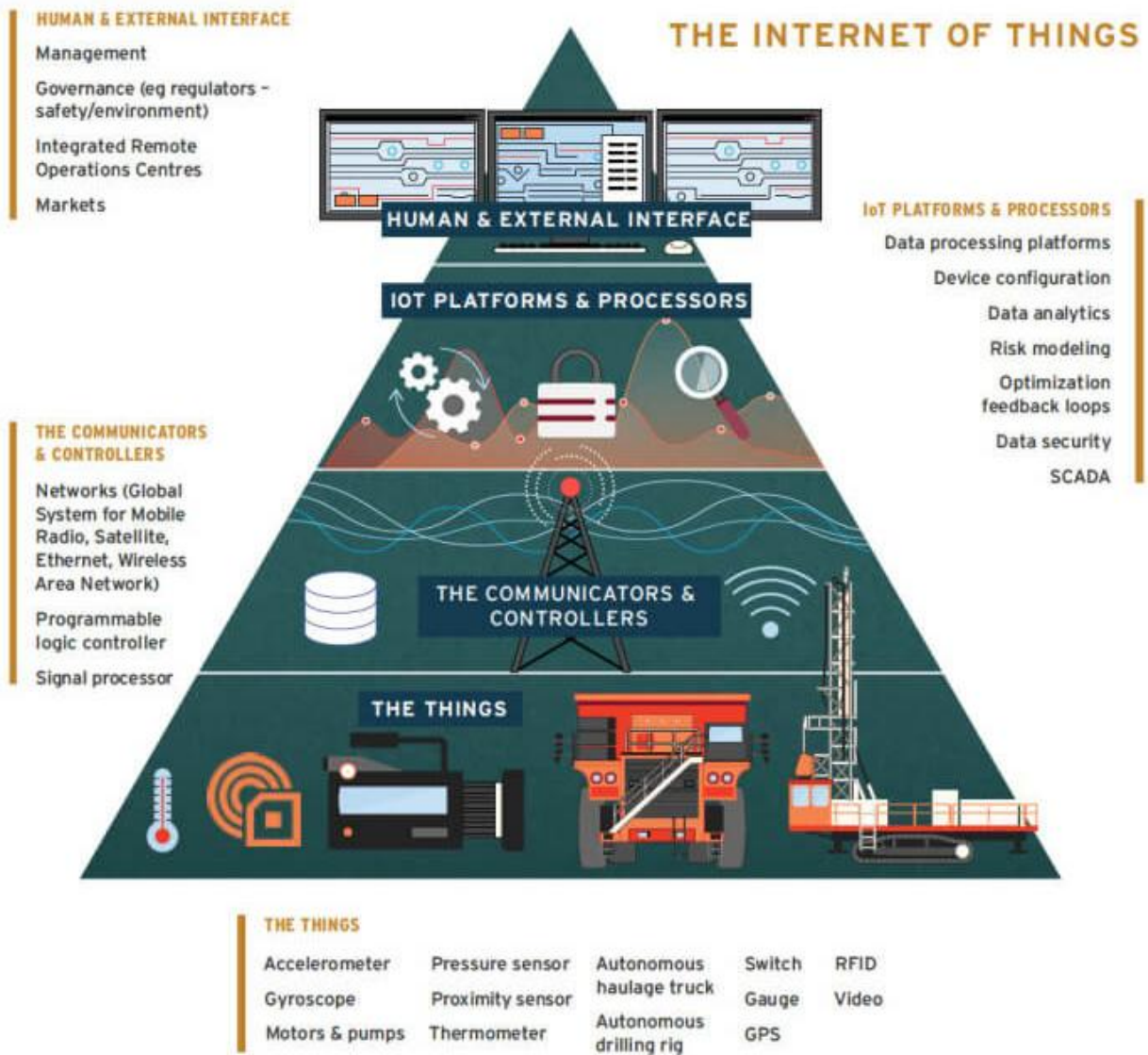
Urbanization	Demographic changes	Changing lifestyles	Climate change
- Urban population worldwide amounts currently to approximate 4.1 billion people - Expected to double by 2050 - Resource depletion; need for efficient management of resources - Exclusion, inequality, and rising insecurity challenges	- Number of seniors aged 60 or over is the fastest growing segment of the population at a rate of 3.26% - Decline in infant mortality & high fertility - Proliferation of the younger population - Need for employment opportunities	- Changes in family patterns - New habits in work and mobility, e.g., tele-working, vehicle sharing, & renting - Need for novel urban services in support of these changes	- Climate changes & global warming - Policies for efficient use of water, energy, and other resources - Measures for sustainable growth

IoT-based future trends for smart homes

This slide mentions the trends to be seen in smart homes operating on IoT technologies. It includes improved voice assistants, smart home security, biometric technologies and energy saving.

- Improved voice assistants**
 - Expand voice control usage in smart showers, refrigerators and door locks
 - Introduction to sound recognition systems for smart devices
 - Add text here
- Smart home security**
 - Ability to erase the recorded data to avoid security breaches
 - Block suspicious activity and notify users of potentially dangerous traffic
 - Add text here
- Biometric technologies**
 - Smart doorbells with built-in facial recognition algorithms
 - Biometric fingerprint door locks to enhance home security
 - Add text here
- Energy saving**
 - Geolocation tracking of users to handle usage of appliances
 - Automatic creation of routine by tracking user habits
 - Add text here





Internet of Things is Changing Healthcare

